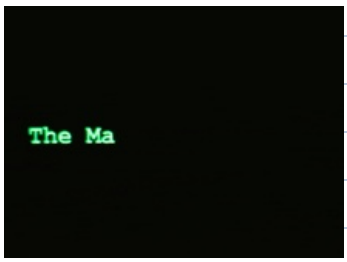


2D Matrices

Song : Boyfriend
- Karan Ayla

Topics To Cover :

1. Basics of Matrix
2. Print row & col wise sum
3. Matrix Diagnols.
4. Server check Question

Hi Everyone!!!

Space Complexity :

Online compiler $\longrightarrow$ memory 4MB $\longrightarrow$ Save upto 10^5 int
$\downarrow$
list of size 10^5

Constraint :

$0 < N*M < 10^4 \longrightarrow$ Can save the matrix
↳ $O(N*M)$ is good.

$0 < N < 10^8 \longrightarrow$ Cannot be saved
↳ len of ar
$\rightarrow O(N)$ not a good SC

Matrix / 2D Array

- List of Lists
- Has rows & cols
- Fixed no. of rows and cols.

	0	1	2	3	4
0	0,0				0,4
1					
2					
3					
4	4,0				4,4

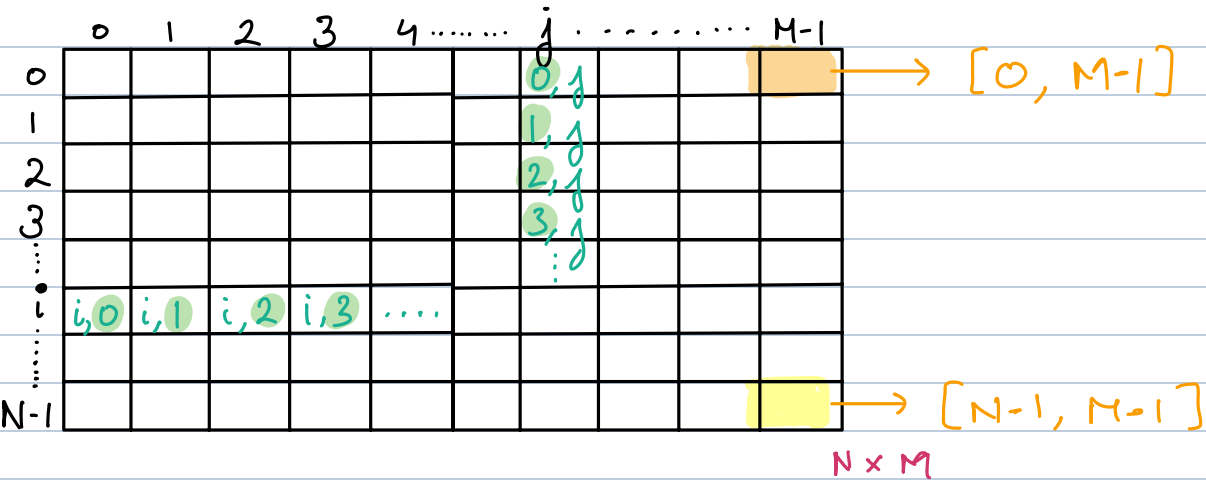
5x5

(row, col)

mat = [[0]*M]*N
or
[[0]*M for _ in range(N)]

eg: $[[0]*3]*2$ → cols → rows

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$



Print the row wise sum

	0	1	2	3	
0	4	3	1	7	→ 15
1	6	2	3	4	→ 15
2	5	3	2	7	→ 17

[15, 15, 17]

```
def row_sum(mat):
    N = len(mat)          (rows)
    M = len(mat[0])       (cols)
    ans = []
    for row in range(N):
        sum = 0
        for col in range(M):
            sum += mat[row][col]
        ans.append(sum)
    return ans.
```

*for SC if a variable is expected to be returned to the user we DO NOT include it in SC

TC: $\Theta(NM)$

SC: $\Theta(N)$ → $\Theta(1)$

Print the col. wise sum:

	0	1	2	3
0	4	3	1	7
1	6	2	3	4
2	5	3	2	7
	15	8	6	18

```
def row_sum (mat):
    N = len(mat)        (rows)
    M = len(mat[0])     (cols)
    ans = []
    for col in range (M):
        sum = 0
        for row in range (N):
            sum += mat[row][col]
        ans.append (sum)
    return ans.
```

TC: $\Theta(N * M)$
SC: $\Theta(1)$

Q. Diagnols of a Sq. Matrix

	0	1	2	3	4
0	0,0	0,1	0,2	0,3	0,4
1	1,0	1,1	1,2	1,3	1,4
2	2,0	2,1	2,2	2,3	2,4
3	3,0	3,1	3,2	3,3	3,4
4	4,0	4,1	4,2	4,3	4,4

5x5

Anti-Diagnol

Principal Diagnol →

r++	c++
r	c
0	0
1	1
2	2
3	3
4	4
5	5

== N break

r++	c--
r	c
0	4 (cols-1)
1	3
2	2
3	1
4	0
5	-1

r<N c ≥ 0

```
r = 0
c = 0
while (r<N and c<N):
    print (mat[r][c])
    r += 1
    c += 1
```

```
r = 0
c = N-1
while (r<N and c≥0):
    print(mat[r][c])
    r+=1
    c-=1
```

TC: N iter for principal diag + N iter for Anti diagnol

$= 2N = \Theta(N)$

SC: $\Theta(1)$

Q. Given a matrix of size $n*m$, print all the diagnols from right to left

↳ Anti-diagnol

	0	1	2	3	4	5
0	10	20	3	5	6	9
1	29	19	12	25	30	15
2	1	5	6	8	9	22
3	21	5	8	1	3	4

4 x 6

o/p:
10
20 29
3 19 1
5 12 5 21
6 25 6 5
9 30 8 8
15 9 1
22 3
4

Q. How many anti diagnols does a matrix of $N*M$ contain?

Break: 10:11 – 10:20

1. Anti Diagnols starting from row 1 = M diagnols

2. Anti Diagnols starting from last col: = N diagnols.

Total Diagnols = $N + M - 1$

↳ Overlap at TR corner

Quiz: Anti diagnols in 4 x 5 matrix

$= 4 + 5 - 1 = 8$ diagnols.

```
def print_diagnols (mat):
    for col in range(M):
        r = 0          // fixed
        c = ~~N-1~~ col
        while (r<N and c≥0):
            print (mat [r][c], end = " ")
            r +=1
            c - =1
        print ()
```

prints anti diagnol from r, c

```
    for row in range (1, N):
        r = ~~0~~ row
        c = M-1        // Fixed
        while (r<N and c≥0):
            print (mat [r][c], end = " ")
            r +=1
            c - =1
        print ()
```

prints anti diagnol from r, c

TC: $\Theta(N*M)$

Q. Mark servers Down :

	t1	t2	t3	t4	t5
S1					
S2					
S3					

for every timeslot t, we get a health check for all the servers s.

* If a healthcheck is ≥1, then it is healthy else if it is 0 its unhealthy. An unhealthy server at any time slot means its unreliable for all timeslots (row) and that timeslot (column) should be considered unstable.

Eg: server ↓ time →

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 0 \\ 9 & 2 & 0 & 4 \end{bmatrix} \longrightarrow \begin{bmatrix} 1 & 2 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

BF: ① Iterate and find 0s, if 0, convert the row and col to -1, skip any other zero

$$\begin{bmatrix} 0 & 1 & 0 \\ 2 & 3 & 1 \\ 5 & 4 & 1 \end{bmatrix} \qquad \begin{bmatrix} 0 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 4 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 3 & -1 \\ -1 & 4 & -1 \end{bmatrix}$$

② Convert all -1s to 0

```
def health_check_status(log):        → list of list
    for r in range(len(log)):
        for c in range(len(log[0])):
            if log[r][c] == 0:
                for i in range(len(log)):
                    if log[i][c] != 0:
                        log[i][c] = -1          } cur col = -1
                for i in range(len(log[0])):
                    if log[r][i] != 0:
                        log[r][i] = -1          } cur row = -1

    for r in range(len(log)):
        for c in range(len(log[0])):            } Converts -1 back to 0
            if log[r][c] == -1:
                log[r][c] = 0
```

$$\begin{bmatrix} 1 & 0 & 1 \\ 2 & 5 & 1 \\ 4 & 0 & 1 \end{bmatrix} \longrightarrow \begin{bmatrix} -1 & 0 & -1 \\ 2 & -1 & 1 \\ -1 & 0 & -1 \end{bmatrix} \longrightarrow \begin{bmatrix} 0 & 0 & 0 \\ 2 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

TC: $\Theta((N*M)(N+M))$

SC: $\Theta(1)$

HW: optimise by saving 0s row and col in a set and finally converting that row or col to 0.